

PatternShop – Bitirme Projesi Tanımı

İçindekiler

1	Genel bakış	1
2	Ne yapacaksınız	2
2.1	Zorunlu özellikler	2
2.2	İş kuralları	2
2.3	Genişletme özellikleri	4
3	Kalıp gereksinimleri	5
4	Önce spesifikasyon: SPEC.md dosyanız	6
4.1	SPEC.md şablonu	6
5	Mimari kurallar	7
6	Size verilenler	7
7	Test beklentileri	7
8	Kabul testlerini çalıştırma	7
9	Teslim edilecekler	8
10	Zaman çizelgesi (9.–14. haftalar)	8
11	Değerlendirme	9
12	Akademik dürüstlük ve yapay zekâ asistanları	9

Ders: Java'da Tasarım Kalıpları (Java 27), Güz 2026 · Süre: 9.–14. haftalar · Spesifikasyonunuzun teslimi: 10. hafta · Sunumlar: 14. hafta · Değerlendirme: [rubrik](#) · English: [spec.en.md](#) · Eğitimciler için mühendislik spesifikasyonu: [SPEC-capstone.md](#)

1 Genel bakış

Modüllerde her kalıbı birkaç sınıfın içinde gördünüz. Bitirme projesinde (capstone) bunları küçük ama eksiksiz tek bir sistemde birleştiriyorsunuz: **PatternShop**, bir çevrim içi mağazanın sipariş işleme çekirdeği. Komut satırında çalışır, tüm verileri bellekte tutar; arayüzü ve veritabanı yoktur. Projeyi ilginç kılan tasarımıdır:

- **önce kendi SPEC.md dosyanızı** yazarsınız (spesifikasyon güdümlü geliştirme, SDD) ve güncel tutarsınız;
- zorunlu özellikleri **verilen bir API'nin** arkasında gerçekleştirirsiniz; böylece yayımlanmış bir **kabul testi takımı** bir özelliğin ne zaman bittiğini size söyler;
- **en az on tasarım kalıbı** kullanır ve **gerekçelendirirsiniz**; ayrıca modern Java 27 ve bir eşzamanlılık kalıbı;
- kodunuz, her derlemede (build) çalışan **ArchUnit kurallarıyla** denetlenen bir **altıgen mimariye** (Ports and Adapters) uyar;
- seçtiğiniz **iki genişletme özelliği** eklersiniz;
- tasarımınızı bir **raporda** anlatır, **kısa bir sunumda** savunursunuz.

Tek bir doğru tasarım yoktur. Kabul testleri *davranışı* denetler; rubrik *iyi gerekçelendirilmiş tasarımı* ödüllendirir. Hiçbir sorunu çözmediği yerde kullanılan bir kalıp puan kaybettirir (m11'deki "patternitis", kalıp hastalığı).

2 Ne yapacaksınız

2.1 Zorunlu özellikler

Kimlik	Özellik	Kısaca
F1	Katalog	Ürün ekleme (fiziksel ya da dijital), SKU ile bulma, kategori / en yüksek fiyat / ada göre arama, stok ekleme
F2	Sepet	Müşteriye sepet açma, ürün ekleme (aynı SKU birleşir), adet değiştirme, çıkarma, kupon uygulama
F3	Geri alma / yineleme (undo/redo)	Sepet düzenlemelerini geri alma ve yineleme; sepet başına, en çok 20 adım geriye
F4	Fiyatlandırma ve promosyonlar	X al Y bedava, kategori yüzde indirimi, eşik üstü tutar indirimi, kuponlar, kargo ücreti — sabit bir sırayla
F5	Doğrulamalı ödeme adımı (checkout)	Tüm sorunları sabit bir sırayla bildiren bir doğrulama kuralları zinciri
F6	Sahte harici API ile ödeme	Garip arayüzlü (metinler, durum kodları) üçüncü taraf tarzı bir API üzerinden tahsilat ve iade
F7	Sipariş yaşam döngüsü	Ödeme adımında PLACED → PAID, sonra SHIPPED → DELIVERED; ödenmiş sipariş iptal edilebilir (iade + stoğa geri)
F8	Olaylar ve bildirimler	Değişiklik kaydedildikten sonra dağıtılan alan olayları (domain event); müşteri bildirimleri; düşük stok uyarıları
F9	Eşzamanlı sipariş karşılama (fulfilment)	Ödenmiş tüm siparişleri sanal iş parçacıklarında (virtual thread), paralel sipariş sınırıyla gönderme
F10	Raporlar	Günlük satış, en çok satanlar, müşteri ekstrezi, envanter — sealed (mühürlü) tipler olarak; metin ve CSV çıktısı
F11	Komut satırı arayüzü	Yukarıdakilerin hepsi için satır tabanlı bir CLI

2.2 İş kuralları

Kabul testlerinin denetlediği kurallar bunlardır. Birebir çıktı metinleri (CLI, raporlar) başlangıç kodunun [test kaynaklarında](#) ve [SPEC-capstone.md](#), "Output formats" bölümündedir. Bu metinle bir test arasında çelişki bulursanız bildirin — testi değiştirmeyin.

Para ve kimlikler. Tüm fiyatlar KDV dahil Türk lirasıdır ve tam kuruş olarak saklanır (Money). 987.91 biçiminde yazdırılır. Yüzdelere, dersteki tüm para tutarları gibi kuruşa **HALF_EVEN** kuralıyla yuvarlanır (Money.percent). Kimlikler her mağaza örneği için üretilir: sepetler `cart-1`, `cart-2`, ...; siparişler `order-1`, `order-2`, ... (bir sipariş numarasını yalnızca gerçekten oluşturulmuş bir sipariş tüketir). Saat (clock) enjekte edilir — `Instant.now()` doğrudan çağrılmaz. Geçersiz girdi (hatalı biçimli SKU, boş ad, pozitif olmayan adet, sepete eklenen bilinmeyen ürün, ...) `IllegalArgumentException` ile; bilinmeyen bir sepet ya da bilinmeyen bir ürüne stok ekleme `NoSuchElementException` ile; kapalı bir sepetteki her düzenleme, geri alma ve yineleme `IllegalStateException` ile reddedilir. Birebir mesajlar GIVEN kullanım senaryolarının Javadoc'undadır.

Katalog (F1). Bir SKU B0K-001 biçimindedir (üç büyük harf, tire, üç rakam). Ad boş olamaz, fiyat pozitif olmalıdır, fiziksel ürünün stoğu ≥ 0 olmalıdır, dijital ürünün stoğu sınırsızdır (0 olarak verilir ve gösterilir). Aynı SKU ikinci kez eklenemez. Arama sonuçları SKU'ya göre sıralıdır. Yalnızca fiziksel ürünlere, pozitif bir miktarla stok eklenebilir.

Sepet (F2). Sepette zaten bulunan bir SKU eklenince adedi artar; satır ilk konumunu korur. Adetler pozitif olmalıdır; adedi 0 yapmak satırı çıkarır. Bilinmeyen ürünler reddedilir ve sepet değişmez. Stok alışveriş sırasında **denetlenmez**, yalnızca ödeme adımında. Bir sepette en çok bir kupon olur; kupon uygulanırken doğrulanır (bilinmeyen ya da süresi dolmuş kupon reddedilir; kupon, saatin saat diliminde `validUntil` günü dahil geçerlidir); geçerli başka bir kupon uygulamak öncekinin yerini alır. Başarılı bir ödeme adımından sonra sepet kapanır ve sonraki her düzenleme reddedilir.

Geri alma / yineleme (F3). Her başarılı düzenleme (ekleme, adet değiştirme, çıkarma, kupon) geri alınabilir; geri alma sepeti satır konumları dahil tam olarak eski haline getirir. Yineleme geri alınan düzenlemeyi yeniden uygular; yeni bir düzenleme yineleme geçmişini siler. Geçmiş sepet başnadır ve son 20 düzenlemeyi tutar. Başarısız düzenlemeler kaydedilmez. Geri alınacak / yinelenecek bir şey yokken geri alma / yineleme `false` döndürür ve hiçbir şeyi değiştirmez.

Fiyatlandırma (F4) — her zaman bu sırayla:

1. Satır toplamı = adet $\times$ birim fiyat; *ara toplam* = satır toplamlarının toplamı.
2. Bir SKU üzerinde **X al Y bedava**: her tam X + Y birimlik grupta Y birim bedavadır.
3. **Kategori yüzde indirimi**: kategorideki her satır için, o satırın 2. adımından sonra kalan tutarının yüzdesi; satır başına, kuruşa **HALF_EVEN** kuralıyla yuvarlanarak (yarımı çifte yuvarlama: tam yarıda kalan tutar çift kuruşa gider: $0.005 \rightarrow 0.00$, $0.015 \rightarrow 0.02$); dersin tamamında para için tek yuvarlama kuralı budur.
4. **Eşik üstü tutar indirimi**: ara toplamdan 2–3. adımların indirimleri çıkarıldığında kalan tutar eşiğe eşit ya da büyükse tutar düşülür. Birden çok promosyon uygunsa yalnızca eşiği en yüksek olan uygulanır.
5. **Kupon**: 4. adımdan sonra kalan tutarın yüzdesi, bir kez **HALF_EVEN** kuralıyla yuvarlanarak. Süresi dolmuş kupon indirim vermez.
6. Ürün toplamı hiçbir zaman 0.00'ın altına inmez: bir indirim kalan tutarla sınırlanır ve 0.00'luk indirimler listelenmez (toplam = ara toplam – indirimler + kargo).
7. **Kargo**: sepette fiziksel ürün varsa ve ürün toplamı 500.00'in altındaysa 49.90; aksi halde 0.00. Toplam = ürün toplamı + kargo.

İndirim etiketleri: `buy 2 get 1 free: TOY-001`, `10% off BOOKS`, `100.00 off over 1000.00`, `coupon AUTUMN5 5%`.

Çözümlü örnek (başlangıç koduyla gelen örnek katalog ve promosyonlar):

Adım	Ayrıntı	Tutar
Sepet	B0K-001 $\times$ 2 (250.00), B0K-002 $\times$ 1 (400.00), TOY-001 $\times$ 3 (120.00), DIG-001 $\times$ 1 (99.90, dijital, BOOKS)	
1 Ara toplam	500.00 + 400.00 + 360.00 + 99.90	1359.90
2 Buy 2 get 1 free: TOY-001	bir birim bedava	–120.00
3 10% off BOOKS	50.00 + 40.00 + 9.99	–99.99
4 100.00 off over 1000.00	1139.91 $\geq$ 1000.00	–100.00
5 Coupon AUTUMN5 5%	1039.91'in %5'i = 51.9955 $\rightarrow$ 52.00	–52.00
7 Kargo	987.91 $\geq$ 500.00	0.00
Toplam		987.91

Ödeme adımı (F5, F6). Doğrulama, herhangi bir tahsilattan önce çalışır. Sepet boşsa tek neden `empty cart` olur. Aksi halde her kural çalışır ve tüm nedenler şu sırayla bildirilir: `missing address` (yalnızca sepette fiziksel ürün varsa; hiçbir alanı boş olmayan adres eksiksizdir) → `quantity limit exceeded: <SKU>` (bir SKU'dan 10'dan fazla birim; sepet sırasıyla) → `insufficient stock: <SKU>` (fiziksel ürünler; sepet sırasıyla) → `expired coupon: <CODE>` → `missing card token`. Geçerli bir sepet harici ödeme API'si üzerinden **tam bir kez** tahsil edilir: tutar metin olarak ("987.91"), para birimi "TRY", ayarlardaki üye işyeri kimliği (merchant id) ve idempotency anahtarı olarak sepet kimliği. Durum 200 → sipariş oluşturulur; 402 → `payment declined`; her 5xx → `payment unavailable`. Reddedilen bir ödeme adımı sipariş oluşturmaz, stoğu değiştirmez, sepeti açık bırakır, olay yayımlamaz ve bildirim göndermez. Oluşturulan sipariş stoğu ayırır (fiziksel ürünler), sepeti kapatır ve saatin zamanı ile sağlayıcının referansı ile `PLACED` ve `PAID` durumlarını kaydeder. Toplamı 0.00 olan sipariş, ödeme API'si çağrılmadan oluşturulur (referans `FREE`).

Sipariş yaşam döngüsü (F7). İzin verilenler: `PAID → SHIPPED` (yalnızca karşılama), `SHIPPED → DELIVERED`, `PAID → CANCELLED`. İptal boş olmayan bir neden ister (yoksa `missing reason`), ödemeyi harici API üzerinden iade eder ve fiziksel ürünleri stoğa geri koyar; iade başarısız olursa sonuç `refund failed` olur ve hiçbir şey değişmez. Diğer her şey `cannot cancel SHIPPED order`, `cannot deliver PAID order`, ... ile reddedilir; bilinmeyen bir kimlik `unknown order: <id>` ile. İş sonuçları istisna değil, sonuçtur (sealed tipler).

Olaylar ve bildirimler (F8). Olaylar (`OrderPlaced`, `OrderPaid`, `OrderShipped`, `OrderDelivered`, `OrderCancelled`, `StockLow`) yalnızca değişiklik kaydedildikten **sonra**, oluştukları sırayla dağıtılır; bu yüzden bir abone yeni durumu görür. Aboneler yalnızca kendi olay tiplerini alır (`ShopEvent` abonesi hepsini alır). İstisna fırlatan bir abone ortamın hata alıcısına bildirilir; diğer aboneler yine çalışır ve işlem yine başarılı olur. Müşteriler (alıcı: müşteri kimliği) `Order order-1 confirmed` (ödemede), `Order order-1 shipped` (takip koduyla) ve `Order order-1 cancelled` (nedeniyle) konulu bildirimler alır. Bir ürünün stoğu en az 5'ten 5'in altına düştüğünde `StockLow` yayımlanır ve `ops` bilgilendirilir — her eşik geçişinde bir kez.

Sipariş karşılama (F9). Tek bir çağrı, o anda `PAID` olan tüm siparişleri gönderir. Sipariş başına sırayla: her fiziksel satırı topla (`pick`), paketle (`pack`), posta koduna gönder (`ship`) — depo API'sine bloklayan çağrılar. Farklı siparişler **sanal iş parçacıklarında** paralel çalışır; aynı anda hiçbir zaman `maxParallelOrders` 'tan (varsayılan 4) fazla değil. Başarısız olan sipariş `PAID` kalır ve istisna mesajıyla bildirilir; diğerleri yine gönderilir. Yalnızca dijital ürün içeren sipariş depoya uğramadan `DIGITAL` takip koduyla gönderilir. Sonuçlar — ve çalıştırma bittikten sonra çağırılan iş parçacığında dağıtılan `OrderShipped` olayları — sipariş numarası sırasındadır (`order-2`, `order-10` 'dan önce). Çağrı ancak tüm iş bittiğinde döner.

Raporlar (F10). Günlük satış (aralığın her günü, sipariş olmayan günler dahil; iptal edilen siparişler hariç), en çok satanlar (birime, sonra SKU'ya göre; liste fiyatlarıyla; iptal edilen siparişler hariç), müşteri ekstresi (tüm siparişler oluşturulma sırasıyla; harcanan toplam iptal edilenleri içermez), envanter (fiziksel ürünler SKU sırasıyla, stok < 5 ise `low`). İstekler ve raporlar sealed tiplerdir; her rapor metin ya da CSV olarak yazdırılır.

CLI (F11). `help` komutları listeler. Her kullanım senaryosuna (use case) CLI'dan ulaşılır; `Main --demo` örnek katalog ve promosyonlarla başlar. Bilinmeyen komutlar `ERROR unknown command: <word>`, hatalı argümanlar komutun `USAGE` satırını yazdırır.

2.3 Genişletme özellikleri

İki tane (ikili çalışmada üç) seçin ve kendi kabul kriterleri ve testleriyle `SPEC.md` dosyanızda tanımlayın. Her biri yalnızca kod değil, gerekçelendirebileceğiniz bir tasarım kararı eklemelidir.

Kimlik	Genişletme	Davet ettiği kalıplar
E1	Paket fiyatlı ürün setleri	Composite (Bileşik)
E2	Satırlarda hediye paketi ve hızlı gönderim seçenekleri	Decorator (Dekoratör)
E3	<code>country = TR and spent >= 1000</code> gibi promosyon uygunluk kuralları	Interpreter (Yorumlayıcı)
E4	CSV ya da JSON satırlarından katalog içe aktarma	Template Method (Şablon Metot), Adapter (Adaptör)
E5	Yeniden başlatmadan sonra da kalan dosya tabanlı depolar (adaptörler yalnızca bileşim kökünde değişir)	Repository (Depo), Adapter
E6	Fiyat düşüşü uyarılı istek listeleri	Observer (Gözlemci), Mediator (Arabulucu)
E7	<code>ServiceLoader</code> ile bulunan sipariş dışı aktarma eklentileri	Factory (Fabrika), eklenti
E8	Ödeme dayanıklılığı: ödeme portu etrafında yeniden deneme ve devre kesici	Decorator, Proxy (Vekil)
E9	İadeler: <code>DELIVERED → RETURN_REQUESTED → REFUNDED</code>	State (Durum)
E10	Yapılandırılmış eşzamanlılık (Structured Concurrency) ile karşılama (önizleme , isteğe bağlı) — m10'daki gibi yalıtılmış; varsayılan yol final API'lerde kalır; verilen tüm testler <code>--enable-preview</code> olmadan geçmelidir	Structured Concurrency
E11	10. hafta spesifikasyon incelemesinde onaylanan kendi fikriniz	—

3 Kalıp gereksinimleri

- Yaratımsal, yapısal, davranışsal ve eşzamanlılık ailelerinden **en az 10 farklı kalıp**; bunların en az 2'si **yaratımsal**, 2'si **yapısal**, 3'ü **davranışsal** ve 1'i **eşzamanlılık** kalıbı olmalıdır. Immutable Object (Değişmez Nesne) eşzamanlılık kalıbınız olarak sayılmaz.
- Mimari kalıplar (Dependency Injection (Bağımlılık Enjeksiyonu), Repository, Specification (Belirtim), Ports and Adapters, alan olayları) mimari gereği zaten zorunludur ve **bu on kalıba sayılmaz**.
- **Modern Java**: kodunuzun record desenleriyle (record pattern) eksiksiz (exhaustive) bir `switch` ile — `default` olmadan — işlediği en az bir `sealed` record hiyerarşisi. Değerler için record'lar, tek metotlu stratejiler için lambda'lar.
- **Eşzamanlılık**: karşılama, paralel sipariş sayısı sınırlı sanal iş parçacıkları kullanır (görev başına iş parçacığı (thread-per-task), Producer–Consumer (Üretici–Tüketici), Guarded Suspension (Korumalı Bekletme)... — seçim ve gerekçe sizin).
- Her katılımcı tipi verilen `@PatternRole` notuyla işaretleyin — kabul takımı kalıpları buradan sayar, değerlendiriciler kodunuzu bununla bulur:

```
// snippet
@PatternRole(value = DesignPattern.STRATEGY, role = "concrete strategy")
```

```
record CategoryPercentOffRule(Category category, int percent) implements PromotionRule
{ ... }
```

- Her kalıp, SPEC.md dosyanızın ve raporunuzun **kalıp gerekçe tablosunda** yer alır (şablon [rubrikte](#)).

4 Önce spesifikasyon: SPEC.md dosyanız

Üretim kodu yazmadan önce `capstone/starter/SPEC.md` dosyasını yazın. 10. haftada (rubrik C1) ve sonda, değişiklik günlüğüyle birlikte yeniden değerlendirilir. Aşağıdaki şablonu kullanın; kısa ve somut tutun — kabul kriterleri denetlenebilir olmalıdır.

4.1 SPEC.md şablonu

```
# PatternShop – <adınız / adlarınız>

> Durum: taslak | onaylı (10. hafta) | son (14. hafta) · Değişiklik günlüğü sonda

## 1. Amaç
Bir paragraf: sistem ne yapar, kimin için. İki ya da üç kullanıcı hikâyesi.

## 2. Kapsam
### 2.1 Zorunlu özellikler
| Kimlik | Özellik | Kabul takımı | Notlarım / yorumum |
### 2.2 Genişletme özellikleri
Her biri için: açıklama, kullanıcı hikâyesi, kabul kriterleri (Given / When / Then),
test sınıfı adları.
### 2.3 Kapsam dışı

## 3. Alan modeli
Çekirdek tiplerin Mermaid sınıf diyagramı (aggregate'ler, değerler, sealed
hiyerarşiler).

## 4. Mimari
Paketler (domain, application, adapter.in, adapter.out, config), giriş ve çıkış
portları,
bileşim kökü. Altıgenin bir Mermaid diyagramı.

## 5. Kalıp planı
| # | Kalıp (aile) | Burada çözdüğü sorun | Katılımcılar | Değerlendirilen alternatif |
Test |

## 6. Eşzamanlılık tasarımı
Ne paralel çalışır, sınır nedir, hatalar ve sonuçlar nasıl toplanır, neden iş parçacığı
güvenlidir.

## 7. Test stratejisi
Verilen kabul testleri, kendi birim testleriniz (hangi test ikizleri), genişletme kabul
testleri.

## 8. Sınırlar ve varsayımlar
Her zaman / Önce sor / Asla. Bu metnin sessiz kaldığı yerlerde yaptığınız varsayımlar.

## 9. Kilometre taşları
10.–13. haftalar için planınız (hafta başına hangi özellikler ve kalıplar).

## 10. Açık sorular ve değişiklik günlüğü
| Tarih | Değişiklik | Neden |
```

5 Mimari kurallar

Kodunuz `io.github.aliturgutbozkurt.patterns.capstone.shop` altında `domain`, `application`, `adapter.in.*`, `adapter.out.*` ve `config` paketlerinde yaşar (Ports and Adapters, m11). Başlangıç kodu, her derlemede çalışan ve iskelet üzerinde zaten yeşil olan yedi ArchUnit kuralıyla gelir. Onları yeşil tutun — notun parçasıdır ve kırmızı bir kural uygulama bölümünü başarısız kılar:

1. Alan katmanı (`domain`) yalnızca JDK'ya (`java.lang`, `java.util`, `java.math`, `java.time`), kendisine ve verilen değer tiplerine, olaylara ve kalıp notlarına bağımlıdır.
2. Uygulama katmanı adaptörlere, `config` paketine ya da harici sistemlerin API'lerine bağımlı değildir.
3. Harici sistemlere (ödeme, depo, bildirimler) yalnızca çıkış adaptörlerinden ve `config` paketinden; simülatlörlere yalnızca `config` paketinden ulaşılır.
4. Adaptörler birbirine bağımlı değildir.
5. Adaptörler yalnızca `config` paketinde (bileşim kökü) bağlanır; hiçbir şey `config` paketine bağımlı değildir.
6. Paketler arasında döngü yoktur.
7. Üretim kodu yalnızca JDK'ya ve ders paketlerine bağımlıdır; `System.exit` yok; değiştirilebilir statik alan yok.

Kendi kurallarınızı ekleyebilirsiniz; verilenleri silemez ya da zayıflatamazsınız.

6 Size verilenler

`capstone/starter` içinde:

- **VERİLEN API** (`...capstone.api`, değiştirmeyin): değer record'ları (`Sku`, `Money`, kimlikler, `Address`), her özellik için bir giriş portu (`CatalogueUseCase`, `CartUseCase`, `PricingUseCase`, `CheckoutUseCase`, `OrderUseCase`, `ShopEvents`, `FulfilmentUseCase`, `ReportUseCase`, `CommandLine`) — hepsi `PatternShop` altında toplanır; sealed istek ve sonuç tipleri; harici sistemler `ExternalPaymentApi`, `WarehouseApi`, `NotificationGateway`; CLI için belirlenimci (deterministic) simülatlörler ve `DemoData`; `@PatternRole` notu.
- **İskelet** (`...capstone.shop`): `config.ShopCompositionRoot` implements `PatternShopFactory` — kabul testlerinin mağazanızı oluşturduğu tek yer — ve `config.Main`, ayrıca açıklama notlu boş paketler.
- **Testler**: 11 takımda 83 kabul testi (`*ExerciseTest`, etiket `exercise`), 7 mimari kural (`ShopArchitectureTest`), test sahteleri (saat, ödeme sanal ortamı (sandbox), betikli depo, kaydeden bildirimler) ve beklenen CLI dökümü ile rapor metinleri.
- **Şablonlar**: `SPEC.md` ve `REPORT.md`.

7 Test beklentileri

- Verilen **83 kabul testinin** ve **7 mimari kuralın** hepsi geçer. Onları asla değiştirmeyin, silmeyin, devre dışı bırakmayın ya da zayıflatmayın; VERİLEN API'yi de asla değiştirmeyin (değerlendiriciler başlangıç koduyla karşılaştırır).
- Alan ve uygulama katmanları için **kendi birim testleriniz**: sayılan her kalıbın davranışını gösteren en az bir testi olur (ör. tek başına bir fiyatlandırma adımı, reddedilen bir durum geçişi, bir komut ve tersi). Elle yazılmış test ikizleri kullanın (m11), mocking kütüphanesi kullanmayın.
- Her genişletme özelliği için, `SPEC.md` içindeki kabul kriterlerine bağlanan **kendi kabul testleriniz**.
- Testler belirlenimcidir: enjekte edilen saat; iş parçacıklarını beklemek için `sleep` yok (m10'daki gibi latch ya da barrier kullanın).
- Kapsam hedefi: `domain` ve `application` için $\geq 80\%$ satır kapsamı (`-Pcoverage`, JaCoCo).

8 Kabul testlerini çalıştırma

Her komutu depo kökünden, JDK 27 ile çalıştırın:

```

export JAVA_HOME=$(/usr/libexec/java_home -v 27)           # macOS; Linux/Windows'ta
JAVA_HOME'u JDK 27'ye yöneltin

./mvnw -q -pl capstone/starter verify                       # derleme + kendi
testleriniz + mimari kurallar (yeşil kalmalı)
./mvnw -q -pl capstone/starter test -Pexercises             # 83 kabul testi (başta
kırmızı)
./mvnw -q -pl capstone/starter test -Pexercises -Dtest='CartExerciseTest' -
Dsurefire.failIfNoSpecifiedTests=false
./mvnw -q -pl capstone/starter verify -Pcoverage           # kapsam raporu: capstone/
starter/target/site/jacoco

./mvnw -q -pl capstone/starter package -DskipTests
java -cp capstone/starter/target/classes
io.github.aliturgutbozkurt.patterns.capstone.shop.config.Main --demo

```

Özellik özellik çalışın: bir takımı yeşile çevirin, commit edin, devam edin. [Zaman çizelgesi \(9.–14. haftalar\)](#) bölümündeki haftalık sıra modülleri izler.

9 Teslim edilecekler

Hepsi ders deposunun kendi çatınızda (fork), `capstone/starter/` altında, `capstone-final` etiketinde:

1. **Kod** — `src/main` ve `src/test`; `./mvnw -q -pl capstone/starter verify` ve `-Pexercises` çalıştırması yeşil.
2. **SPEC.md** — spesifikasyonunuz ([SPEC.md şablonu](#)), son değişiklik günlüğüyle.
3. **Testler** — kendi birim testleriniz ve genişletme kabul testleriniz (bkz. [Test beklentileri](#)).
4. **REPORT.md** — 6–10 sayfa: altıgen diyagramıyla tasarım özeti, kalıp gerekçe tablosu, modern Java ve eşzamanlılık kararları, neyi değiştirdiniz, yapay zekâ kullanım beyanı. İngilizce **ya da** Türkçe yazılır; diğer dilde bir sayfalık özet içerir (terimler `docs/glossary.md`'ye göre).
5. **Sunum** — 14. haftada 10 dakika + 5 dakika soru: canlı CLI gösterimi, üç kalıbın derinlemesine anlatımı, yaptığınız bir ödünleşim (trade-off). Slaytlar `presentation.pdf` olarak.

10 Zaman çizelgesi (9.–14. haftalar)

Hafta	Ders modülü	Bitirme projesi çalışması	Kilometre taşı
9	m07 Observer, Chain, ...	Metni okuyun, çatalayın, başlangıç kodunu çalıştırın, m11 Ports and Adapters bölümünü önden okuyun; <code>SPEC.md</code> taslağı; genişletmeleri seçin	M0 başlangıç kodu derleniyor; <code>verify</code> yeşil
10	m08 State, sealed tipler	<code>SPEC.md</code> 'yi bitirin; katalog, sepet, geri alma	M1 — SPEC.md teslimi (notlanır, bir hafta içinde geri bildirim)
11	m09 veri odaklı	Fiyatlandırma (Strategy + Decorator), sipariş yaşam döngüsü (State), doğrulama (Chain)	M2 Catalogue, Cart, Undo, Pricing takımları yeşil
12	m10 eşzamanlılık	Ödeme adımı Facade'ı, ödeme adaptörü, olaylar ve bildirimler	M3 Checkout, Lifecycle, Events takımları yeşil
13	m11 mimari	Eşzamanlı karşılama, raporlar, CLI, genişletmeler, kendi testleriniz	M4 83 testin ve 7 kuralın hepsi yeşil (özellik dondurma)

Hafta	Ders modülü	Bitirme projesi çalışması	Kilometre taşı
14	Sunumlar	Rapor, slaytlar, sunum saatiniz- den önce <code>capstone-final</code> etike- ti	M5 son teslim, sunum ve savun- ma

9.–13. haftalar arasında her hafta en az bir commit yapın — geçmiş, işin size ait olduğunun kanıtlarından biridir (rubrik C8). `SPEC.md` 10. haftadan sonra değişebilir; her değişikliği nedeniyle birlikte değişiklik günlüğüne yazın.

11 Değerlendirme

100 puan, ayrıntısı [rubrikte](#): spesifikasyon ve tasarım 25 (C1–C2), uygulama ve testler 50 (C3–C8), rapor ve savunma 25 (C9–C10). Ders izlencesinde bitirme projesi dersin notunun %40'ıdır (spesifikasyon %10 · uygulama ve testler %20 · rapor ve savunma %10).

Uygulama bölümünün otomatik başarısızlığı (C3–C8 = 0): derleme ya da kendi testleriniz JDK 27'de başarısız; verilen herhangi bir kabul testi kırmızı; herhangi bir mimari kural kırmızı; ya da verilen bir test, kural veya VERİLEN API tipi değiştirilmiş, silinmiş, devre dışı bırakılmış veya zayıflatılmış. Rubrik bu koşulların tamamını listeler.

12 Akademik dürüstlük ve yapay zekâ asistanları

- Eğitmeniniz 9. haftada bir ikiliyi onaylamadıysa bitirme projesi **bireysel** bir çalışmadır.
- Ders **modüllerindeki** kodu (örnekler ve kendi ödev çözümleriniz) yeniden kullanabilirsiniz; bunu `// adapted from modules/m07-.../EventBus.java` gibi bir yorumla belirtin. Belirtilmeyen yeniden kullanım intihaldir.
- Depoda bir **referans çözüm** de vardır (`capstone/reference`). Ondan ya da başka bir öğrenciden kopyalamak akademik dürüstlük ihlalidir: uygulama bölümü 0 ile notlanır ve durum kurumun prosedürüne göre ilerler. Teslimler referans çözümle ve birbirleriyle karşılaştırılır.
- Yapay zekâ asistanları izlencedeki kurallarla serbesttir: neler için kullandığınızı `REPORT.md` içinde beyan edin ve her satırı açıklamaya hazır olun — savunma (rubrik C10) kendi kodunuz hakkında sorular sorar. `SPEC.md` , kalıp seçimleri ve gerekçeleri size ait olmalıdır.